

Kiste v0.9.1 Spec

Standard Config Contract: kiste.yaml, kiste.lock, .kiste/, Bootstrap, and Target

One-sentence spec: Kiste v0.9.1 standardizes kiste.yaml, kiste.lock, and .kiste/ as the official project contract, version lock, and generated state/report directory for Kiste-managed repos and workspaces.

This release should come after v0.9.0. Version 0.9.0 fixes the library, plugin, static module, and split-ready architecture. Version 0.9.1 defines the stable config surface that users actually touch.

1. Main Goal

Kiste needs a standard project file, similar in role to package.json for Node projects, Dockerfile for container build recipes, go.mod for Go modules, and terraform.lock.hcl for Terraform provider locking. The Kiste standard should be simple, boring, and predictable.

kiste.yaml -> human-owned project/workspace config
kiste.lock -> exact Kiste/module/plugin/schema version lock
.kiste/ -> generated scan reports, graphs, manifests, cache, and plans

Rule: kiste.yaml says what the project is. .kiste/ says what Kiste found. kiste.lock freezes the versions used to interpret and generate it.

2. File Roles

File	Owner	Purpose
kiste.yaml	Human + Kiste-assisted	Main project/workspace contract
kiste.lock	Kiste generated	Freezes Kiste, schema, module, and plugin versions
.kiste/spec.json	Kiste generated	Normalized scan result
.kiste/graph.json	Kiste generated	Workspace dependency graph
.kiste/modules.json	Kiste generated/configurable	Enabled module state
.kiste/doctor-report.json	Kiste generated	Repo/workspace health result
.kiste/bootstrap-plan.json	Kiste generated	Planned bootstrap changes before writing
.kiste/release-check.json	Kiste generated	Release readiness result
.kiste/blobs/*.manifest.json	Kiste generated	Metadata-first blob manifests

3. Version Header

Every kiste.yaml should start with an explicit version, schema identifier, and kind.

```
kiste: "0.9.1"  
schema: "kiste.config/v0.9.1"  
kind: Project
```

Allowed kind values:

Project
Workspace
Plugin
Template

Most user repos use kind: Project. Multi-repo systems use kind: Workspace.

4. Minimal kiste.yaml

A simple backend repo should need only a short file.

```
kiste: "0.9.1"
schema: "kiste.config/v0.9.1"
kind: Project

project:
  name: robot-api
  type: service
  description: Robot API backend

runtime:
  language: python
  framework: fastapi
  entrypoint: main:app
  start: uvicorn main:app --host 0.0.0.0 --port 8000

container:
  dockerfile: Dockerfile
  image: docker.io/kistebox/robot-api:latest
  ports:
    - 8000

env:
  required:
    - DATABASE_URL
    - SECRET_KEY
  optional:
    - LOG_LEVEL

target:
  preferred: kubernetes
  alternatives:
    - cloud-run
    - azure-container-apps

modules:
  enabled:
    - repo
    - tokens
    - workspace
    - gitops
    - secrets

bootstrap:
  plan_before_apply: true
  write_requires_confirm: true
```

This is enough for kiste doctor, kiste scan ., kiste bootstrap plan, and kiste target plan.

5. Workspace kiste.yaml

A workspace config describes a multi-repo product/system.

```
kiste: "0.9.1"
schema: "kiste.config/v0.9.1"
kind: Workspace

project:
  name: robot-platform
  type: workspace
  description: Multi-repo robot software platform
  owner: KisteBox
  lifecycle: alpha

modules:
  enabled:
    - repo
    - tokens
    - workspace
    - gitops
    - secrets
    - images
    - targets
    - data
    - blob
    - cloud
    - argocd

workspace:
  repos:
    - name: frontend
      type: frontend
      provider: github
      repo: KisteBox/robot-frontend
      path: ./frontend

    - name: api
      type: backend
      provider: github
      repo: KisteBox/robot-api
      path: ./api

    - name: worker
      type: worker
      provider: github
      repo: KisteBox/robot-worker
      path: ./worker

  connections:
    - from: frontend
      to: api
      relation: calls

    - from: api
      to: postgres
      relation: uses-db

    - from: worker
      to: queue
      relation: consumes
```

6. Standard Top-Level Sections

kiste.yaml should support these top-level sections. Not every repo needs every section. Unknown sections should fail validation unless explicitly allowed.

```
kiste:
schema:
kind:

project:
```

```
modules:
workspace:
runtime:
services:
container:
dependencies:
env:
secrets:
tokens:
images:
data:
blob:
gitops:
argocd:
cloud:
target:
bootstrap:
release:
```

7. Full Standard Config Example

```
kiste: "0.9.1"
schema: "kiste.config/v0.9.1"
kind: Workspace
```

```
project:
  name: robot-platform
  type: workspace
  description: Multi-repo robot software platform
  owner: KisteBox
  lifecycle: alpha
```

```
modules:
  enabled:
    - repo
    - tokens
    - workspace
    - gitops
    - secrets
    - images
    - targets
    - data
    - blob
    - cloud
    - argocd
```

```
services:
  - name: api
    type: web
    language: python
    framework: fastapi
    entrypoint: main:app
    start: uvicorn main:app --host 0.0.0.0 --port 8000
    port: 8000
    container:
      dockerfile: ./api/Dockerfile
      image: docker.io/kistebox/robot-api:latest

  - name: frontend
    type: web
    language: node
    framework: vite
    start: npm run dev
    port: 5173
    container:
      dockerfile: ./frontend/Dockerfile
      image: docker.io/kistebox/robot-frontend:latest

  - name: worker
    type: worker
    language: python
    start: python worker.py
    container:
```

```

    dockerfile: ./worker/Dockerfile
    image: docker.io/kistebbox/robot-worker:latest

dependencies:
  databases:
    - name: postgres
      type: postgres
      env: DATABASE_URL

  caches:
    - name: redis
      type: redis
      env: REDIS_URL

  queues:
    - name: queue
      type: redis
      env: CELERY_BROKER_URL

env:
  required:
    - DATABASE_URL
    - REDIS_URL
    - SECRET_KEY
  optional:
    - LOG_LEVEL
    - SENTRY_DSN

secrets:
  backend: sops
  files:
    - secrets/api-secret.enc.yaml
    - secrets/db-secret.enc.yaml
  policy:
    allow_plaintext_env_files: false
    require_encrypted_kubernetes_secrets: true

tokens:
  - name: github-dev
    provider: github
    scopes:
      - repo:read
      - packages:write
    expires_at: 2026-08-01
    secret_reference: env://KISTE_TOKEN_GITHUB_DEV

images:
  - name: api-image
    service: api
    image: docker.io/kistebbox/robot-api:latest
    require_digest_pin: false

data:
  datasets:
    - name: weld-detection
      provider: roboflow
      ref: workspace/project/1
      task_type: object-detection
      format: yolov8
      blobs:
        - weld-detection-export

blob:
  refs:
    - name: weld-detection-export
      provider: s3
      uri: s3://kiste-datasets/weld-detection/v1/
      version: v1
      policy:
        download_by_default: false
        allow_sample: true
        max_sample_files: 20
        max_sample_bytes: 10485760

```

```
gitops:
  output: ./gitops
  environments:
    - dev
    - prod
  default_environment: dev
  generator:
    kubernetes: true
    network_policy: true
    secrets: true

argocd:
  enabled: true
  namespace: argocd
  project: kiste-platform
  repo_url: https://github.com/KisteBox/kiste-gitops.git
  sync_policy:
    self_heal: true
    prune: false

cloud:
  provider: aws
  region: ap-southeast-1
  kubernetes: eks
  cluster:
    name: kiste-dev
    environment: dev
  mode: plan-only

target:
  default: kubernetes
  rules:
    prefer_serverless_for:
      - stateless-http
      - low-traffic
    prefer_kubernetes_for:
      - gpu
      - stateful
      - long-running-worker
      - service-mesh

bootstrap:
  default_level: workspace
  plan_before_apply: true
  write_requires_confirm: true

release:
  channel: alpha
  require:
    tests: true
    docs: true
    no_plaintext_secrets: true
    no_large_blobs_in_git: true
```

8. kiste.lock

kiste.lock is machine-generated. It records the exact Kiste version, schema version, plugin API version, and module versions used for the project.

```
{
  "kiste": "0.9.1",
  "schema": "kiste.config/v0.9.1",
  "plugin_api": "0.1",
  "generated_at": "2026-05-30T00:00:00Z",
  "modules": {
    "repo": "0.9.1",
    "tokens": "0.9.1",
    "workspace": "0.9.1",
    "gitops": "0.9.1",
    "secrets": "0.9.1",
    "images": "0.9.1",
    "targets": "0.9.1",
    "data": "0.9.1",
    "blob": "0.9.1",
    "cloud": "0.9.1",
    "argocd": "0.9.1"
  }
}
```

```
kiste module lock --write
kiste module verify
```

9. Generated .kiste/ Layout

```
.kiste/
  spec.json
  graph.json
  modules.json
  doctor-report.json
  bootstrap-plan.json
  release-check.json

  blobs/
    weld-detection-export.manifest.json

  self/
    self-check.json
    self-review.md
    release-plan.md
```

Recommended .gitignore policy:

```
.kiste/cache/
.kiste/tmp/
.kiste/logs/
```

Recommended to commit: kiste.yaml and kiste.lock. Optional to commit: .kiste/blobs/*.manifest.json and .kiste/release-check.json. Usually do not commit cache, temp, logs, or local JSON state.

10. Bootstrap vs Target

Bootstrap is the onboarding/change process. It answers: what files should Kiste add or update so this repo becomes Kiste-managed?

Target is the deployment destination plan. It answers: where and how should the service/workspace run?

```
doctor = what is wrong?
target = where should it run?
bootstrap = what files/changes are needed to prepare it?
generate = create specific artifacts
apply = write approved changes
```

Recommended flow:

```
kiste doctor
kiste target plan
kiste bootstrap plan
kiste bootstrap apply --confirm
kiste bootstrap validate
```

Rule: target chooses the deployment destination. bootstrap prepares the repo for that destination.

11. Commands for v0.9.1

Config commands:

```
kiste init
kiste config validate
kiste config print
kiste config schema
kiste config migrate
```

Bootstrap process:

```
kiste bootstrap plan
kiste bootstrap apply --confirm
kiste bootstrap validate
kiste bootstrap status
kiste bootstrap report
```

Doctor, target, lock, and blob commands:

```
kiste doctor
kiste doctor --json
kiste target plan
kiste module lock --write
kiste module verify
kiste blob manifest weld-detection-export
```

12. Validation Rules

kiste config validate should check:

```
[OK] valid schema version
[OK] known kind
[OK] project name exists
[OK] enabled modules are known
[OK] service names are unique
[OK] env var names are valid
[OK] token secret_reference uses env:// or approved secret backend
[OK] blob refs do not download by default
[OK] cloud mode is plan-only unless admin mode is enabled
[OK] Argo CD prune is false by default
```

Validation failures should be explicit:

```
[FAIL] plaintext token found
[FAIL] unknown top-level section
[FAIL] duplicate service name
[FAIL] blob download_by_default set to true without admin approval
[FAIL] cloud mode apply without admin mode
```

13. What v0.9.1 Should Not Do

Version 0.9.1 should not add new product features. It standardizes config, lockfile, generated output layout, validation rules, and migration path.

```
No new cloud providers
No new data providers
No remote plugin install
No automatic deployment
No automatic Argo CD sync
No automatic large downloads
No automatic secret rotation
```

No marketplace

14. Acceptance Criteria

v0.9.1 is complete when these commands work:

```
kiste init
kiste config validate
kiste config print
kiste config schema
kiste bootstrap plan
kiste doctor
kiste module lock --write
kiste module verify
```

Required outputs:

```
[OK] kiste.yaml created
[OK] kiste.yaml validated
[OK] schema exported
[OK] bootstrap plan written
[OK] doctor report written
[OK] kiste.lock written
[OK] module versions verified
[OK] .kiste output layout standardized
```

15. Final v0.9.1 Rule

Kiste v0.9.1 should make kiste.yaml, kiste.lock, and .kiste/ boring, stable, documented, and predictable.

That is the config foundation Kiste needs before adding more providers, deployment actions, marketplaces, or runtime automation.